# Homework 2: CNN Design, Regularization, and Optimization on MNIST

**Objective.** Design and implement your own convolutional neural network (CNN), then investigate how architecture, regularization, and optimization affect generalization. The provided notebook may be used for MNIST loading and preprocessing, but you are responsible for implementing the model, training procedure, experiments, plots, and analysis.

> ***Important:*** *This homework is not a copy-and-modify exercise. Your baseline CNN must be your own implementation. Keep experimental settings consistent when making controlled comparisons.*

## Part 1 - Build Your Own Baseline CNN

Construct a CNN for MNIST digit classification satisfying all of the following:

• At least **2 convolutional layers**.

• At least **1 pooling layer**.

• At least **1 fully connected hidden layer**.

• Use ReLU or another appropriate nonlinear activation.

• Use a 10-class softmax output layer.

• Train the baseline using **SGD**.

• Do not use dropout or explicit L1/L2 regularization in the baseline.

You must choose the number of filters, kernel sizes, pooling locations, hidden-layer size, learning rate, batch size, and number of epochs. Include **model.summary()** and briefly justify your major design choices.

## Part 2 - Tensor Shapes and Parameter Counts

Determine the output tensor shape and number of trainable parameters after each major layer. Include a table similar to the following (add/remove rows to match your architecture):

| Layer | Output Shape | Trainable Parameters |
|---|---|---|
| Input | ... | 0 |
| Conv1 | ... | ... |
| Pooling | ... | 0 |
| Conv2 | ... | ... |
| Hidden Dense | ... | ... |
| Output | ... | ... |

For at least **one convolutional layer** and **one dense layer**, manually show how you calculated the number of trainable parameters. Your calculations must agree with model.summary().

## Part 3 - Establish the Baseline

Train your baseline CNN using a fixed training/validation split. Record:

- Training loss and validation loss.
- Training accuracy and validation accuracy.
- Best validation accuracy and the epoch at which it occurs.

Create two figures: (1) training and validation accuracy versus epoch and (2) training and validation loss versus epoch. Based on the curves, discuss whether the model shows evidence of underfitting, appropriate fitting, or overfitting.

## Part 4 - Regularization Experiments

Starting from your baseline architecture, conduct the following controlled experiments. Change the regularization-related component while keeping other settings as consistent as possible:

• **Dense-layer dropout**.

• **Convolutional feature-map dropout** using SpatialDropout2D.

• **L1 regularization**.

• **One additional regularization method of your choice** (for example L2, data augmentation, or another justified method).

For each of the first three methods, choose the regularization strength yourself. Test at least **two candidate strengths** (for example two dropout rates or two L1 coefficients), report the results, and justify the setting selected for your final comparison. You must also decide where in the network the regularization should be applied.

## Part 5 - Optimization Experiment

Compare the SGD baseline with **Adam**. Choose reasonable learning rates for both optimizers. Do not rely only on optimizer defaults without explanation. Compare convergence speed, stability of the learning curves, best validation accuracy, and final validation accuracy. Discuss whether faster initial convergence necessarily produced the best final model.

## Part 6 - Architecture Experiment

Modify the CNN architecture itself. Choose **one** meaningful modification, such as:

• Add another convolutional block.

• Increase or decrease the number of filters.

• Replace Flatten() with GlobalAveragePooling2D().

• Change kernel sizes or pooling strategy.

• Another meaningful architecture modification, with a clear explanation.

State your hypothesis before reporting the result: explain why you expected the modification to improve or reduce performance. Compare the modified architecture with the original baseline.

## Part 7 - Final Comparison

Create one figure showing **validation accuracy versus epoch** for at least the following: baseline, best dense-dropout model, best convolutional-dropout model, best L1 model, Adam model, and architecture-modified model.

| Model | Parameters | Optimizer | Regularization | Best Val. Acc. | Best Epoch |
|---|---|---|---|---|---|
| Baseline | ... | SGD | None | ... | ... |
| Dense Dropout | ... | SGD | ... | ... | ... |
| Conv Dropout | ... | SGD | ... | ... | ... |
| L1 | ... | SGD | ... | ... | ... |
| Adam | ... | Adam | None | ... | ... |
| Architecture Variant | ... | ... | ... | ... | ... |

## Part 8 - Error Analysis

Using your best-performing model:

• Compute and display a confusion matrix on the test set.

• Identify the **two digit classes most frequently confused**.

• Display at least **six misclassified test images**, including true and predicted labels.

• Briefly discuss visual characteristics that may have contributed to these errors.

## Part 9 - Written Analysis

1. Which modification produced the largest improvement over your baseline?

2. Did dropout reduce the gap between training and validation performance? Support your answer with results.

3. How did L1 regularization affect validation performance and training behavior?

4. How did Adam differ from SGD in convergence behavior?

5. Did increasing or changing model complexity necessarily improve generalization?

6. Based on your experiments, which CNN configuration would you choose and why?

## Submission Requirements

• Submit your completed Jupyter notebook (.ipynb) with all code and outputs.

• Submit a PDF report containing the requested tables, figures, calculations, and written analysis. The report may be generated from your notebook if it is clearly organized and readable.

• Your notebook must run in a reasonable order and reproduce the reported experiments.

• Clearly label each experiment and identify the hyperparameters used.

• Do not submit only final accuracy numbers; experimental evidence and analysis are required.

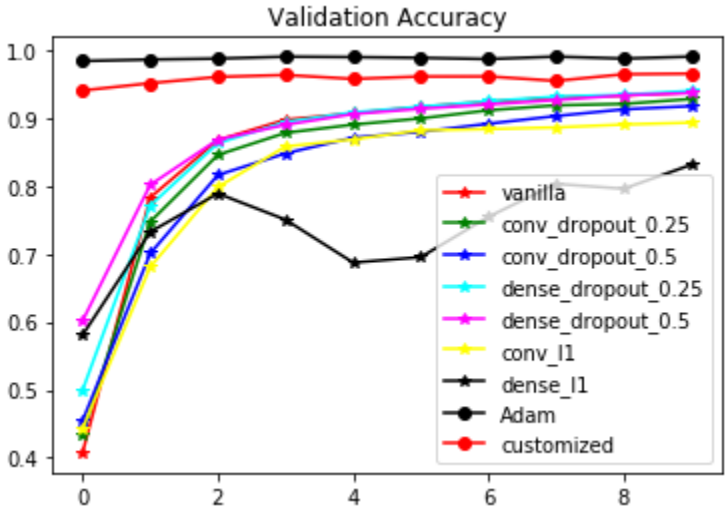
## Grading Emphasis

| Category | Weight |
|---|---|
| CNN implementation and experimental design | 40% |
| Analysis and technical reasoning | 30% |
| Results, tables, and figures | 20% |
| Code quality and reproducibility | 10% |

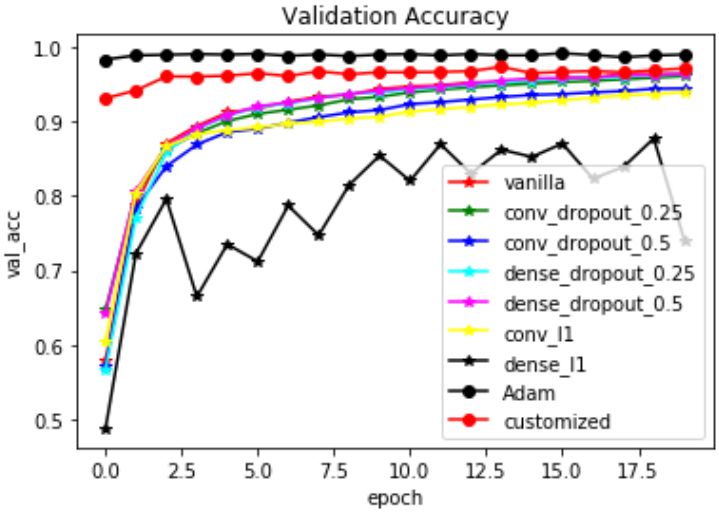
**Academic integrity:** You may consult TensorFlow/Keras documentation, but the submitted implementation, experiments, figures, and analysis must represent your own work. Follow all course academic-integrity requirements.

# Example Experimental Validation Figures

*The following figures are examples of how validation-accuracy curves may be presented. They are provided only as a formatting and comparison reference. Your models, hyperparameters, training curves, and numerical results must come from your own experiments.*




**Example 1.** Validation accuracy comparison across multiple CNN variants over 10 training epochs.




**Example 2.** Validation accuracy comparison across multiple CNN variants over 20 training epochs.

*For your submission, make your figure readable: label the axes, include a legend, use consistent experiment names, and ensure that all required models can be compared clearly. You may use a different visual style.*